Daily Quant Digest: 2026-11-05

Foundations Restart, day 10. Three new steps today: the most famous puzzle in probability, **Monty Hall**, and the real lesson inside it (the *rule* that produced the news matters as much as the news); the **yield curve** and how US Treasuries are quoted (a menu of borrowing prices, and prices written in 32nds); and **binary search**, the "halve it again" trick that finds anything in a sorted list of a million items in 20 looks.

Today's topics

Track	Topic	Difficulty
Probability & Statistics	FR Lesson 10 (part 1 of 2): classic puzzles: Monty Hall, and why the host's rule matters	Easy-Medium
Derivatives	FR Lesson 10: the yield curve; US Treasuries and how they are quoted	Easy-Medium
Algorithms & DS	FR Lesson 10: binary search: first and last position of a price	Easy-Medium
Music	Pink Floyd, "Comfortably Numb" (1979)	

1. Probability & Statistics: Monty Hall, or the Host Who Knows

> **Where we are.** Lesson 6: news **moves the walls in**, and $P(A \mid B)$ is the share of the new small room taken by A. Lesson 8: split into cases and add the paths of a tree. Lesson 9 (yesterday): Bayes' rule, "given what I saw, which case was I in?", and its fast odds form: new odds = old odds $\times$ likelihood ratio. Today we use all three on a puzzle that fooled thousands of maths professors. Part 2 (tomorrow): the birthday problem and boy-or-girl.

1.1 The game

A game show has three closed doors. Behind one is a car. Behind the other two are goats. You pick a door, say door 1. The host, **who knows where the car is**, opens one of the other two doors and shows you a goat, say door 3. He then asks: "Do you want to stay with door 1, or switch to door 2?"

Almost everyone says: “Two doors left, so it is 50/50. Switching makes no difference.” That answer is wrong. Switching wins **2 times out of 3**.

1.2 The central picture: a guide who must avoid the treasure

Think of the host as a **guide who knows where the treasure is and is forbidden to reveal it**. When he opens a door, he is not a random gust of wind blowing a door open. He is choosing, and his choice is forced most of the time.

In the picture	In the maths
The guide who knows the treasure	The host, who knows the car’s door
“Forbidden to reveal it”	He always opens a goat door, never your door
A forced choice	When you picked a goat, only one door is left for him to open
A free choice	When you picked the car, he can open either goat door
What he opened	The news B we condition on

Here is the key scene. You point at door 1.

- **If door 1 hides a goat** (2 times in 3), the car is behind door 2 or door 3. The guide cannot open your door and cannot open the car door. He has exactly **one** door he is allowed to open. The door he leaves shut is the car. Switching wins.
- **If door 1 hides the car** (1 time in 3), he can open either other door. Switching loses.

So switching wins exactly when your first pick was wrong, which is 2 times in 3. The host’s opening did not change the chance that your first pick was right. It just **pointed at** the car in all the games where you were wrong.

A bigger picture makes it obvious. Take 100 doors. You pick door 1. The host, who knows, opens 98 goat doors and leaves door 1 and door 57 shut. Would you stay? Your door had a 1 in 100 chance when you picked it, and nothing about that has changed. Door 57 is the one door the guide carefully stepped around. Switching wins 99 times in 100.

1.3 Count first: 300 games

Imagine playing 300 times. You always pick door 1. The car is placed at random.

Car behind	Games	What the host does	Games where he opens door 3
Door 1	100	Free choice: opens door 2 or 3, half each	50
Door 2	100	Forced: must open door 3	100
Door 3	100	Forced: must open door 2	0

Now the news arrives: **he opened door 3**. The walls move in (Lesson 6). The new room holds $50 + 100 = 150$ games. In 100 of them the car is behind door 2.

$$P(\text{car at } 2 \mid \text{opens } 3) = \frac{100}{150} = \frac{2}{3}, \qquad P(\text{car at } 1 \mid \text{opens } 3) = \frac{50}{150} = \frac{1}{3}.$$

Read it out loud: "Among all the games where the host opened door 3, two thirds had the car behind door 2."

1.4 The same thing with odds (Lesson 9)

Before the host moves, door 1 and door 2 are equally likely: prior odds $1 : 1$.

Now ask for each case: how likely was the news "he opened door 3"?

- Car at 1: he chose freely, so $P(\text{opens } 3) = \frac{1}{2}$.
- Car at 2: he was forced, so $P(\text{opens } 3) = 1$.

Likelihood ratio (door 2 vs door 1) $= 1 \big/ \frac{1}{2} = 2$. New odds $= 1 : 1 \times 2 = 2 : 1$ in favour of door 2. Probability $2/3$. Same answer, three lines.

Notice what made the evidence point at door 2: the host was **twice as likely** to open door 3 when the car was behind door 2. A forced action is stronger evidence than a free one.

1.5 The real lesson: the rule matters, not just what you saw

Change one thing. The host **does not know** where the car is. He opens door 2 or door 3 at random, and it happens to show a goat. Same thing you saw on the stage. Different answer.

Count 300 games again. He opens door 3 in half of every group, and sometimes that reveals the car:

Car behind	Games	He opens door 3	Of those, door 3 shows a goat
Door 1	100	50	50
Door 2	100	50	50
Door 3	100	50	0 (the car is revealed, game over)

Given “he opened door 3 and it was a goat”, the room holds $50 + 50 = 100$ games, half with the car at door 1 and half at door 2. Now it really is **50/50**.

Why the change? Compare the “car at 2” row. The knowing host is **forced** to open door 3 in all 100 of those games, because door 2 hides the car. The ignorant host opens door 3 in only 50 of them, because he is just flipping a coin. So for the knowing host, “he opened door 3” is twice as likely when the car is at door 2 (likelihood ratio 2). For the ignorant host it is equally likely either way (likelihood ratio 1), and the odds stay $1 : 1$.

That is the whole lesson in one sentence: **to update correctly, you must know how the news was produced, not only what it says.** Traders meet this every day. A broker who “happens to mention” a big buyer is a different piece of evidence from a broker whose job is to show you only the flows that make you trade.

1.6 Common mistakes

- **“Two doors left, so 50/50.”** Equally likely is only true when nothing breaks the symmetry. The host’s forced choice breaks it.
- **Conditioning on the wrong event.** The news is not “door 3 has a goat”. The news is “the host, following his rule, chose to open door 3”. Those two can have different probabilities, and only the second gives 2/3.
- **Forgetting the host’s free choice.** In the “car at 1” row he opens door 3 only half the time. Dropping that $\frac{1}{2}$ is the most common slip in the algebra.

1.7 Self-check

1. Five doors, one car. You pick one. The host, who knows, opens three goat doors. Should you switch, and what is your chance if you do?
2. In 300 standard games where you always switch, about how many cars do you win?
3. A friend played the standard game: he picked door 1 and the host opened door 3. You walk in late and see only two closed doors. You do not know which one your friend picked, so you choose one of them at random. What is your chance of winning?

1.8 Interview question: four doors and different hosts

(A Jane Street / SIG style warm-up. The interviewer is checking whether you condition on how the information was produced.)

Four doors, one car, three goats. You pick door 1.

a. The host, who knows, opens one goat door among the other three (choosing at random if he has a choice). You switch to one of the two remaining closed doors at random. What is your chance of winning?

b. Instead, the host who knows opens **two** goat doors. You switch to the last closed door. Chance of winning?

c. Now the host **does not know**. He opens one of doors 2, 3, 4 at random and it happens to show a goat. What is the chance the car is behind door 1? Behind each of the other two closed doors? Should you switch?

d. Back to **three** doors and a host who knows, but he is lazy: whenever door 2 hides a goat, he always opens door 2. You pick door 1 and he opens **door 3**. What is the chance that switching wins? What if he had opened door 2?

1.9 Solution

(a) Switching wins only if your first pick was wrong (chance $3/4$) **and** you then pick the right door among the two left (chance $1/2$, since the car is behind one of them):

$$P(\text{win}) = \frac{3}{4} \times \frac{1}{2} = \frac{3}{8} = 37.5\%.$$

Count to check with 400 games: the car is not at door 1 in 300 of them; in each, the host leaves the car closed plus one goat, and you pick the car in half: 150 wins out of $400 = 3/8$. Staying wins only $1/4$, so switching is better even though you are not sure which door to switch to.

(b) Now the host leaves only the car shut whenever you were wrong. Switching wins exactly when your first pick was wrong: $3/4$.

(c) Count 1,200 games (so every number below is whole). The car is at each door in 300 games. The host picks one of doors 2, 3, 4 at random.

- Car at door 1 (300 games): whatever he opens is a goat. 300 games survive.
- Car at door 2, 3 or 4 (900 games): he hits the car with chance $1/3$; a goat with chance $2/3$. 600 games survive.

Given "a goat was shown", the room holds $300 + 600 = 900$ games. The car is at door 1 in 300 of them: $P = 1/3$. The other 600 are split equally between the two other closed doors: $1/3$ each.

So all three closed doors have $1/3$. **Switching neither helps nor hurts.** Compare with the knowing host in (a): there door 1 stayed at $1/4$ and each other door rose to $3/8$.

(d) Write out the lazy host's rule for each car position (you picked door 1):

Car behind	Door 2 a goat?	Host opens
Door 1	yes	door 2 (his favourite)
Door 2	no	door 3 (forced)
Door 3	yes	door 2 (his favourite)

He opens door 3 **only** when the car is behind door 2. So if you see door 3 open, switching wins with probability **1**.

If he had opened door 2, the car is at door 1 or door 3, equally likely (one game each in the table): switching wins **1/2**. On average over both cases: $\frac{1}{3} \times 1 + \frac{2}{3} \times \frac{1}{2} = \frac{2}{3}$, the same as the standard game. The lazy rule does not change the average, it changes what each opening tells you.

Interview trick. For any "knowing host" game, ask one question: *"When does switching win?"* The answer is usually "when my first pick was wrong", which gives the probability in one line. For hosts with unusual rules, write the small table of "car here, host does what" and count.

Sanity checks. (a) $3/8 + 3/8 + 1/4 = 1$: the three closed doors share all the probability. (c) $1/3 \times 3 = 1$. (d) The two cases average back to $2/3$. All fractions were checked by exact enumeration on a computer.

Self-check answers. (1) Switch: you win whenever your first pick was wrong, $4/5$. (2) About 200. (3) You pick door 2 (wins $2/3$) or door 1 (wins $1/3$) with half chance each: $\frac{1}{2} \cdot \frac{2}{3} + \frac{1}{2} \cdot \frac{1}{3} = \frac{1}{2}$. Not knowing which door your friend picked throws away the host's information.

---

2. Derivatives: The Yield Curve, and How Treasuries Are Quoted

> **Where we are.** Lesson 1: compounding, $(1+r)^t$. Lesson 6: no-arbitrage, "two roads to the same village must cost the same". Lesson 8: a bond's price is the present value of its payments; price and yield sit on a seesaw. Lesson 9 (yesterday): duration and DV01, how hard that seesaw swings. Today we zoom out: there is not one interest rate but a whole **line** of them, one for each length of time.

2.1 The scene

The US government borrows money for many different lengths of time. It sells **US Treasuries**:

Name	Life when issued	How it pays
Bills	4 weeks up to 1 year	No coupon. Bought below 100, repaid at 100.
Notes	2, 3, 5, 7, 10 years	Fixed coupon every six months, 100 at the end
Bonds	20 and 30 years	Same as notes, just longer

Who buys what? A money market fund parks cash in 3-month bills. A Hong Kong bank's treasury holds 2-year notes as safe, liquid assets. A US pension fund that must pay pensions in 2055 buys 30-year bonds. Every one of these buyers sees a different yield, and

if you plot the yield against the length of the loan you get the **yield curve**. It is the single most-watched picture in finance, because every other rate (mortgages, swaps, FX forward points) is priced off it.

2.2 The central picture: a menu of lease prices

Think of a landlord renting out a flat. He offers a menu: the rent per year if you sign for 1 year, for 2 years, for 10 years. He can charge different yearly rents for different lease lengths.

In the picture	In the maths
The landlord's menu	The yield curve
Lease length	Maturity (time to repayment)
Rent per year on that lease	The yield for that maturity
Expecting rents to rise next year	Expecting the central bank to raise rates
Extra charge for being locked in for long	The **term premium**
Today's 1-year lease rent	The short end, close to the central bank's policy rate

Why would the 2-year lease cost more per year than the 1-year one? Two reasons.

1. **Expectations.** If everyone expects rents to rise next year, the landlord will not lock in today's low rent for two years. He asks for a blend of this year's rent and next year's expected rent. In bonds: if the market expects the Fed to raise rates, 2-year yields sit above 1-year yields.
2. **Being locked in.** A long lease means neither side can walk away if the world changes. Investors usually want a little extra to lend for long, because long bonds swing hard (yesterday: the 10-year had about three times the DV01 of the 3-year). That extra is the term premium.

Where the analogy stops: a landlord sets his menu. The yield curve is set by millions of buyers and sellers trading every second.

2.3 What the menu says about the future: the forward rate

Say the 1-year yield is 4.00% and the 2-year yield is 4.50% (both with annual compounding, as in Lesson 1). Two roads take USD 1 to the end of year 2 (Lesson 6):

- **Road A:** lend for 2 years at 4.50%: $1.045^2 = 1.092025$.
- **Road B:** lend for 1 year at 4.00%, then roll for one more year at some rate f: $1.04 \times (1 + f)$.

No-arbitrage says the two roads must cost the same, so

$$1+f=\frac{1.045^2}{1.04}=\frac{1.092025}{1.04}=1.050024, \qquad f\approx 5.00\%.$$

Read it out loud: "the curve is quietly saying that one-year money, one year from now, costs about 5%." This f is the **forward rate**. A rule of thumb: $2\times 4.50-4.00=5.00$. The 2-year rate is (roughly) the average of this year's 4% and next year's 5%.

2.4 Three shapes

Shape	Picture	Usual reading
Normal (upward)	Long leases cost more per year	Steady economy, plus term premium
Flat	Same rent for every lease	Market unsure, often near the end of hikes
Inverted (downward)	Short leases cost more than long	Market expects rate **cuts**, often because it expects a slowdown

Inverted example: 1-year at 5.00%, 2-year at 4.50%. Then $1+f=1.045^2/1.05=1.040024$, so the forward is about **4.00%**: the market expects rates to fall by a full point.

Traders summarise the shape with a **spread** in basis points (1 bp = 0.01%). The famous one is **2s10s**: 10-year yield minus 2-year yield. If the 2-year is 3.50% and the 10-year is 4.10%, 2s10s is $+60$ bp. A trade that profits when this number rises is a **steepener**; when it falls, a **flattener**. Yesterday's DV01 hedge, which lost USD 120,137 when the curve steepened, was a flattener without meaning to be.

Link to FX. In Lesson 6, forward points came from the gap between two currencies' interest rates. Each tenor (1M, 3M, 1Y) uses the rate for that length from each currency's curve. So the shape of the USD and EUR curves is exactly the shape of the EURUSD forward points curve.

2.5 How Treasuries are quoted

Notes and bonds: price in 32nds. The price is per 100 of face value, and the fraction after the dash is in **32nds** (a habit from the old days of halving and halving again).

Quote	Meaning	Decimal
98-16	$98+16/32$	98.5
98-17+	$98+17.5/32$ (the "+" adds half a 32nd)	98.546875
101-08+	$101+8.5/32$	101.265625

One 32nd is called a **tick**. On USD 10m face, one tick is worth $10{,}000{,}000\times\frac{1}{100}\times\frac{1}{32}=$ USD 3,125. Yields are quoted alongside the price, usually to three decimals (for example 4.125%). The quoted price leaves out interest that has built up since the last coupon; that piece is added on settlement and gets its own lesson later.

Bills: on a discount rate. A bill pays 100 at maturity and nothing else. Its quote is a **discount rate** d, using a 360-day year:

$$\text{Price} = 100 \times \left(1 - d \times \frac{\text{days}}{360}\right).$$

Read it out loud: "start at 100, and knock off the yearly discount, scaled down to the bill's life." The discount is taken on the 100 you get back, not on the money you pay, so the true return is a bit higher than d:

$$\text{money market yield} = \frac{100 - \text{Price}}{\text{Price}} \times \frac{360}{\text{days}}.$$

2.6 Worked example: a morning on the desk

Step 1: buy a 13-week bill. USD 10m face, 91 days, quoted at a discount rate of 4.00%.

- Price $= 100 \times (1 - 0.04 \times 91/360) = 100 \times (1 - 0.0101111) = 98.988889$.
- Cash paid: USD 9,898,888.89. Cash back in 91 days: USD 10,000,000. Interest earned: USD 101,111.11.
- Money market yield $= \frac{1.011111}{98.988889} \times \frac{360}{91} = 4.0409\%$, a little above the quoted 4.00%, as promised.

Step 2: buy a 10-year note. USD 10m face at 98-17+.

- Decimal price: $98 + 17.5/32 = 98.546875$.
- Cash (before accrued interest): $10{,}000{,}000 \times 0.98546875 = \text{USD } 9{,}854{,}687.50$.

Step 3: the market moves. By the afternoon the note trades at 98-25. That is $25 - 17.5 = 7.5$ ticks up.

- Profit: $7.5 \times 3{,}125 = \text{USD } 23{,}437.50$.

Step 4: link to DV01. Yesterday a 10-year 4% bond had a DV01 of about USD 8,111 per 10m. In ticks, $8{,}111/3{,}125 \approx 2.6$. So a **1 bp fall in yield is worth about 2.6 ticks** on a 10-year. Our 7.5-tick gain means the 10-year yield fell by about $7.5/2.6 \approx 3$ bp. Traders do this conversion in their heads all day.

2.7 When the curve made headlines: the 2022 to 2024 inversion

In 2022 the Fed raised rates very fast to fight inflation. Short yields shot up, and in July 2022 the 2-year yield rose above the 10-year: 2s10s went negative. It stayed inverted for about two years, until around September 2024, the longest inversion on record. An inverted curve had come before most US recessions, so many forecasters called one. It did not arrive in that window. The lesson: the curve is the market's **expectation** (plus a term premium), not a promise. Banks that bought long bonds in 2020 and 2021 and then watched yields jump also learned how painful the seesaw can be (SVB, in Lesson 8).

2.8 Likely interview questions

1. **"The curve is inverted. What is the market telling you?"** That it expects short rates to fall, usually because it expects the central bank to cut into a slowdown. Forward rates sit below today's short rate.
2. **"The 1-year yield is 3% and the 2-year is 4%. What is the 1-year rate in one year?"** $1.04^2/1.03 - 1 = 5.01\%$. Quick check: $2 \times 4 - 3 = 5$.
3. **"Convert 101-08+ to decimal and give its value on USD 50m."** 101.265625; $50{,}000{,}000 \times 1.01265625 = \text{USD } 50{,}632{,}812.50$.

---

3. Algorithms & Data Structures: Binary Search

> **Where we are.** Lesson 1: Big-O, and why $\log n$ is tiny. Lesson 6: sorting, so we can now assume a sorted list. Lesson 7: two pointers, which used the fact that sorted data lets you throw away part of the list safely. Lesson 9 (yesterday): fast power, which reached x^n by **halving** n each step. Binary search halves the **list** each step.

3.1 The central picture: the fence in a field

Picture a long row of fence posts. Every post from the left is painted **red** up to some point, then every post is **green** to the end. You want the first green post. You are blindfolded, but you can ask "what colour is post number i?"

You could walk from the left and check every post. Or: check the **middle** post. If it is green, the first green is here or to the left, so forget the right half. If it is red, the first green is to the right, so forget the left half including this post. Every question halves the field.

In the picture	In the code
Row of posts	Indices $0 \dots n-1$
Red, red, …, green, green	A **yes/no question** that is false, false, …, true, true
The first green post	The answer we return
The part of the field still in play	The range $[lo, hi)$
Checking the middle post	`mid = lo + (hi - lo) / 2`

The whole art of binary search is to **phrase the problem as a red/green fence**. Once you can do that, the same ten lines solve it.

How many questions? Each one halves what is left. From n posts you get to 1 after about $\log_2 n$ halvings. For $n = 1{,}000{,}000$ that is 20 (because $2^{20} = 1{,}048{,}576$). The test program counts them: 20.

3.2 The problem

You are given the trade prices of a day, sorted in **non-decreasing** order (the same price can appear many times), and a target price. Return the index of the **first** and **last** trade at exactly that price. If it never traded, return `[-1, -1]`.

- $0 \le n \le 10^5$, prices and target are any `int`.
- Required: $O(\log n)$ time.

prices	target	answer
[101, 102, 102, 102, 105, 107, 107, 110]	102	[1, 3]
same	107	[5, 6]
same	104	[-1, -1]
[7, 7, 7, 7]	7	[0, 3]
[]	5	[-1, -1]

3.3 Brute force first

Walk left to right. The first time you see the target, write down its index as `first`. Every time you see it, overwrite `last`. That is $O(n)$ time and $O(1)$ space, simple and correct. It ignores the one gift we were given: **the list is sorted**. The test file keeps this version to check the fast one.

A half-way idea: binary search to find **any** copy of the target, then walk left and right to the ends. That fails the requirement in the worst case: if all 10^5 prices equal the target, the walk is $O(n)$.

3.4 The insight: two fences

Ask the fence question about each post i: **"is `prices[i] >= target`?"** For target 102:

index	0	1	2	3	4	5	6	7
price	101	102	102	102	105	107	107	110
$\ge$ 102?	no	**yes**	yes	yes	yes	yes	yes	yes
$>$ 102?	no	no	no	no	**yes**	yes	yes	yes

Because the list is sorted, each question really is a fence: all "no" then all "yes". So:

- `first` = the first post where "$\ge$ target" is yes. Here 1.
- The first post where "$>$ target" is yes is just **past** the last copy. Here 4, so `last` $= 4 - 1 = 3$.
- If the first "$\ge$" post is off the end, or holds a different price, the target never traded.

Two binary searches, each $O(\log n)$. **Time $O(\log n)$, space $O(1)$.**

Trace for "≥ 102" (lo = 0, hi = 8):

lo	hi	mid	prices[mid]	≥ 102?	action
0	8	4	105	yes	hi = 4
0	4	2	102	yes	hi = 2
0	2	1	102	yes	hi = 1
0	1	0	101	no	lo = 1
1	1				stop, return 1

3.5 Why it never gets stuck: the invariant

Most binary search bugs are off-by-one loops that never end. We avoid them with one promise that stays true at every step:

> **The first green post is somewhere in** $[lo, hi]$, where $hi = n$ means "none of them".

- At the start, $lo = 0$, $hi = n$: true.
- If `mid` is green, the first green is at `mid` or left of it: set `hi = mid` (keep `mid`).
- If `mid` is red, the first green is right of it: set `lo = mid + 1` (drop `mid`).
- Since $lo \leq mid < hi$, either move makes the range strictly smaller, so the loop must end. When $lo = hi$, the promise says that is the answer.

And `mid = lo + (hi - lo) / 2` instead of `(lo + hi) / 2`: if `lo` and `hi` are both near 2^{31}, their sum overflows an `int`. Today's indices are small, but the habit costs nothing and matters in "binary search on the answer" (next lesson), where `lo` and `hi` are values, not indices.

3.6 C++ solution

```cpp
#include <algorithm>
#include <vector>
using namespace std;

// pred is false, ..., false, true, ..., true over [lo, hi).
// Return the first index where it is true, or hi if none.
template <typename Pred>
int firstTrue(int lo, int hi, Pred pred) {
    while (lo < hi) {                    // the answer lives in [lo, hi]
        int mid = lo + (hi - lo) / 2;    // no overflow, and mid < hi
        if (pred(mid)) {
            hi = mid;                    // mid might be the first true: keep it
        } else {
            lo = mid + 1;                // mid is false: answer is to its right
        }
    }
    return lo;                           // lo == hi: the fence post
}

vector<int> searchRange(const vector<int>& prices, int target) {
    const int n = static_cast<int>(prices.size());

    int first = firstTrue(0, n, [&](int i) { return prices[i] >= target; });
```

```
    if (first == n || prices[first] != target) {
        return {-1, -1};                // target never traded
    }
    int afterLast = firstTrue(0, n, [&](int i) { return prices[i] > target; });
    return {first, afterLast - 1};
}

// Same answer using the standard library.
vector<int> searchRangeSTL(const vector<int>& prices, int target) {
    auto [lo, hi] = equal_range(prices.begin(), prices.end(), target);
    if (lo == hi) return {-1, -1};
    int first = static_cast<int>(lo - prices.begin());
    int last  = static_cast<int>(hi - prices.begin()) - 1;
    return {first, last};
}
```

The full file `2026-11-05.cpp` adds the brute force, the examples, a 200,000-case random test against both the brute force and the STL version, values at the `int` limits, and a step counter for one million prices. It compiles cleanly with `g++ -std=c++17 -O2 -Wall` and all tests pass.

3.7 Walkthrough of the C++

`template <typename Pred>`. A template is a recipe the compiler fills in for each type you use it with. Every lambda has its own unnamed type, so we cannot write that type down. `Pred` lets `firstTrue` accept **any** callable: a lambda, a function, anything you can call with `pred(mid)`. Because the compiler knows the exact type, it can inline the call, so this is as fast as writing the comparison by hand. (The alternative, `std::function<bool(int)>`, also works but adds a small cost per call.)

The lambda `[&](int i) { return prices[i] >= target; }`. A small unnamed function written in place. The `[&]` part is the **capture list**: it says "see the variables around me (`prices`, `target`) by reference", so nothing is copied. The `(int i)` is its parameter, and the body is one comparison. Changing `>=` to `>` is the only difference between the two searches. That is the payoff of separating "how to search" (`firstTrue`) from "what to ask" (the lambda).

`const vector<int>& prices`. A reference to the caller's vector (Lesson 2): no copy of 10^5 numbers, and `const` promises we will not change it.

`static_cast<int>(prices.size())`. `size()` returns an unsigned type (`size_t`). Mixing unsigned and signed numbers is a classic trap (Lesson 7: `size() - 1` on an empty vector is a huge number, not -1). We convert once, on purpose, and work with `int` from then on.

`return {-1, -1};`. The braces build the `vector<int>` return value directly from a list. The same trick builds `{first, afterLast - 1}`.

`first == n || prices[first] != target`. The order matters. `||` stops as soon as the left side is true, so when `first == n` we never read `prices[n]`, which would be past the end (undefined behaviour). This is called **short-circuit evaluation**.

`equal_range` and iterators. The standard library already has this fence search:

Function	Returns an iterator to
`lower_bound(b, e, x)`	the first element $\geq x$
`upper_bound(b, e, x)`	the first element $> x$
`equal_range(b, e, x)`	both, as a pair

An **iterator** is a "pointer-like bookmark" into the container: `prices.begin()` points at the first element, `prices.end()` points **one past** the last, the same half-open $[lo, hi)$ idea as our code. Subtracting two iterators of a vector gives the distance between them, so `lo - prices.begin()` turns a bookmark back into an index. (`std::distance(prices.begin(), lo)` does the same and also works for containers like `list`.) These functions require a sorted range; on an unsorted one they return nonsense without any error.

`auto [lo, hi] = ...`. A structured binding (Lesson 7): it unpacks the returned pair into two named variables in one line.

The test file. `mt19937 rng(20261105)` is a random number generator with a fixed seed, so every run makes the same "random" tests and a failure can be reproduced. For each test it builds a small sorted vector with many repeats (only up to 10 different values), picks a target that is sometimes missing or below or above every price, and uses `assert` to check that all three versions agree. Small values with many duplicates are exactly where off-by-one bugs hide.

Common binary search bugs (all avoided above):

- `while (lo <= hi)` with `hi = mid`: when $lo = hi$ this loops forever.
- `lo = mid` instead of `lo = mid + 1`: with $hi = lo + 1$, `mid` equals `lo` and nothing moves.
- Forgetting the "not found" check and reading `prices[n]`.
- Searching an unsorted list.

Self-check. (1) In `[2, 4, 4, 4, 9]`, what do `lower_bound` and `upper_bound` return for 4, as indices? How many 4s are there? (2) How many questions does binary search need, at most, for $n = 100{,}000$? (3) Phrase "the last trade with price $\leq x$" as a fence question.

Answers. (1) 1 and 4; $4 - 1 = 3$ copies. In general the count of x is `upper_bound - lower_bound`. (2) $2^{16} = 65{,}536 < 100{,}000 \leq 2^{17} = 131{,}072$, so 17. (3) Find the first index with `price > x` (the fence), then step back one; if that first index is 0, no trade was $\leq x$.

---

4. Music: Pink Floyd, "Comfortably Numb" (1979)

Context

"Comfortably Numb" is on **The Wall**, Pink Floyd's double album released at the end of 1979. It is credited to **David Gilmour** (music) and **Roger Waters** (words). Waters sings the verses and Gilmour sings the choruses, so the song is a conversation between the

band’s two great voices. It came out as a single in 1980.

The story behind it

Gilmour wrote the music, including the chorus melody, while working on his solo album in 1978 and kept it back. Waters then built it into The Wall, the story of a rock star named Pink who walls himself off from the world. In the song, a doctor is getting Pink, sick and drifting, ready to go on stage. Waters has said the idea came from a night on tour in 1977 when he was ill and was given an injection so he could get through the show, and spent the concert feeling far away from his own body.

The recording was also a fight. Gilmour wanted a harder, rawer version; Waters and producer **Bob Ezrin** liked a lusher one with an orchestra arranged by **Michael Kamen**. The final record is a compromise built from both. That tension between the band’s two leaders runs through the whole album, and The Wall was one of the last records the classic line-up made together.

Why people love it

Two reasons: the contrast and the solos. The verses are dark, slow and heavy, with Waters’ cold, almost spoken voice. Then the chorus opens up, Gilmour’s voice floats in, and the chords suddenly turn bright and warm, like light coming through a window. And then there are the **two guitar solos**. The first is short and sad. The second, which ends the song, is regularly voted among the greatest guitar solos ever recorded. Gilmour has said he assembled it from several takes, choosing the best phrase from each.

What to listen for

- **The two worlds.** Notice how the verse chords feel heavy and shut-in, and how the chorus lifts into a brighter key. The song keeps moving between the doctor’s cold room and Pink’s dream.
- **Gilmour’s string bends.** He pushes a string up to reach a note slowly, so each note seems to cry or sigh rather than just start. Listen for how few notes he plays, and how long he lets each one ring.
- **The orchestra.** Under the choruses, strings swell quietly. It is easy to miss on first listen, but it is a big part of the dreamy feeling.
- **The final solo’s build.** It starts calm and melodic, climbs higher, and then becomes fast and fierce before the song fades away while he is still playing. Notice that it never really “ends”: it just drifts out, like Pink himself.

Tomorrow: Lesson 10 part 2 (the birthday problem and boy-or-girl), repo (borrowing against bonds), and binary search on the answer.